

Social Compliance

Scribe: Yuyan Wang, Zeqiang Ning

1/29/2025

1 Introduction

Social compliance in robotics is a crucial concept that extends beyond traditional paradigms of safety and liveness. While safety ensures that the system does not enter hazardous states and liveness guarantees eventual task completion, social compliance focuses on behavior that aligns with human expectations and societal norms. A socially compliant robot is expected to navigate, interact, and function in ways that are perceived as natural and acceptable by humans.

What makes this property somewhat challenging in terms of safety and liveness is that it relies less on control and more on perception and reasoning, which heavily depend on computer vision and AI toolkits.

1.1 Observation Functions

From a robot-robot to a robot-human scenario, the observation function is different. The observation function is often overloaded owing to human’s facial expression, gesture, pose, body language, etc.

- In a robot-robot setting, the observation function is

$$O_t^i = \{x_t^i\} \cup \{x_t^j, j \in N(x_t^i)\}. \quad (1)$$

- However, in a robot-human setting, the observation function should integrate additional data sources such as RGB images, LiDAR scans, and depth information.

$$O_t^i = O_t^i \cup \boxed{\text{RGB}}, \boxed{\text{Lidar}}, \boxed{\text{Depth}}. \quad (2)$$

Therefore, a higher-level perception function is required to process the complex data. In other words, rather than directly applying the policy to the observations to compute the action, we must first pass the observations through a black box. The overall decision-making process in social compliance follows a structured pipeline:

$$x_t^i \rightarrow o_t^i \rightarrow \boxed{\text{Black box}} \rightarrow \pi_t^i \rightarrow u_t^i \quad (3)$$

So we’ll start to look at some approaches, and understand how the algorithm evaluate social navigation. But more importantly, as a research area, what are some of the aspects we need to look at moving forward.

2 VLM-Social Nav

The first approach that we are looking for is called VLM-Social Nav, introduced in a 2024 research paper. This paper is modeled as an MDP (Single agent), but it still embodies this framework:

$$\langle x, u, T, J \rangle \quad (4)$$

Where

$$\begin{aligned}
x: x_t^i &= \{b_x, b_y, \theta\} \\
u: u_t &= \{v, \omega\} \\
T: x \times u &\rightarrow x \\
v^*, \omega^* &= \operatorname{argmin}_{(v, \omega) \in U^i} \text{ (optimal linear and angular velocity)} \\
J &= \alpha J_{goal} + \beta J_{safety} + \gamma J_{social}
\end{aligned}$$

This approach utilizes a basic unicycle dynamics model combined with a differential drive kinematic model. The key improvement is in its cost function J . We already discussed J_{goal} and J_{safety} in the previous lecture, where $J_{goal} = \|x_t^i - x_g^i\|$ represents the liveness cost and $J_{safety} = \|x_t^i - obs_t\|$ represents the safety cost. The new term J_{social} represents the cost of social compliance. In VLM-Social Nav, we define it as:

$$J_{social} = \|\beta^i - \beta^h\|. \quad (5)$$

- β^i is a navigation behavior, which can be trajectories Γ^i or sequence of control Ψ^i . Both are the same for deterministic system, but usually they are not.
- β^h is a navigation behavior humans would adopt in social conventions. We typically learn it from human expert data $D = \{\Gamma^h, \Psi^h\}$.

We can solve Equation 5 using IRL, Behavior Cloning, and other supervised approaches. In VLM-Social Nav, Equation 5 is addressed using VLM. Figure 3 emphasizes the process that VLM-Social Nav find the optimal action (linear velocity v^* and angular velocity ω^*).

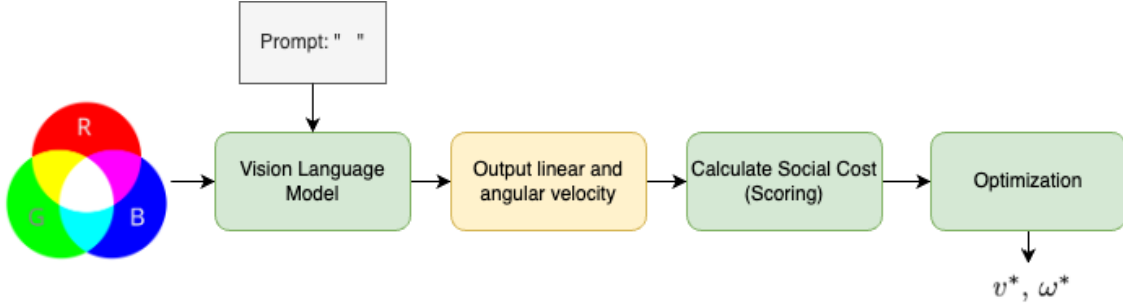


Figure 1: Overall Process of VLM-Social Nav

VLM is very important in VLM-Social Nav in inferring socially compatible navigation behavior β_{t+1}^h based on its pre-trained large internet-scale dataset. An RGB image and a prompt are fed into VLM:

$$\beta_{t+1}^h = VLM(I_t, P, u_t^i) = \{v_{t+1}^h, \omega_{t+1}^h\} \quad (6)$$

According to the VLM, the control inputs $\{v_{t+1}^h, \omega_{t+1}^h\}$ are the one that the robot should take in order to be socially compliant. Therefore,

$$J_{social}^{t+1} = \|\beta - \beta^h\| = \|v - v_{t+1}^h\| + \|\omega - \omega_{t+1}^h\| \quad (7)$$

Given all the cost terms, the low-level optimizer (based motion planner) finds the action $\{v^*, \omega^*\}$ that minimizes the cost. One important consideration is that you cannot simply set the next control to be v_{t+1}^h and ω_{t+1}^h . Other factors, such as safety, must also be addressed.

The overall pipeline of VLM-Social Nav is:

$$x_t^i \rightarrow o_t^i \rightarrow \boxed{\text{VLM}} \rightarrow \pi_t^i \rightarrow u_t^i \quad (8)$$

3 Learn from Large Dataset

A more traditional approach is that we can learn the human behavior parameter, β^h , from data as trajectories. This method relies on data to predict human actions and reactions, allowing the robot to conform to expected social behaviors.

Assume you have collected a dataset of human expert trajectories:

$$J_{social} = ||E_{\Gamma_\theta \sim D_\theta} \Phi(\Gamma_\theta) - E_{\hat{\Gamma} \sim \hat{D}} \Phi(\hat{\Gamma})||. \quad (9)$$

Where

$\Phi()$: some feature function that scores a trajectory

θ : the weight of a Neural Network

D_θ : the dataset that built from trajectories that are generated by the Neural Network

$\hat{D}$: the human expert trajectories dataset

We train through a Neural Network and average the feature score to minimize the difference between Γ_θ and $\hat{\Gamma}$. We update θ until $\Gamma_\theta^i \approx \hat{\Gamma}^i$

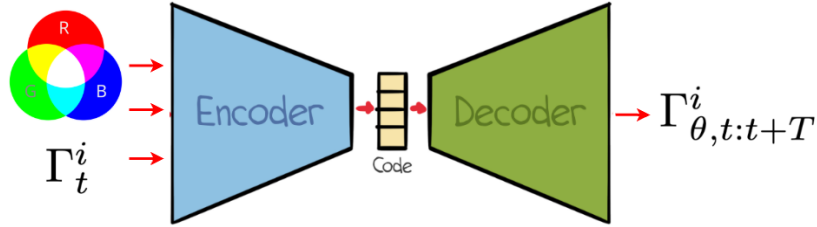


Figure 2: Overall process of training dataset using encoder

The overall pipeline is:

$$x_t^i \rightarrow o_t^i \rightarrow \boxed{Encoder} \rightarrow u_t^i \quad (10)$$

4 Costmap layering

In the previous method, we ensured that the robot's trajectory closely resembled human behavior by training the controller. Here, there is another approach: training the cost function itself to achieve this goal.

4.1 Move Base

In ROS (Robot Operating System), Move Base is a package used for robot navigation. It includes:

- **Global Planner:** Responsible for planning the optimal path from the start to the goal.
- **Local Planner:** Adjusts the trajectory in real-time based on the global plan and dynamic obstacles.
- **Other Stuff**

4.2 Costmap

It worth noting that Global Planner can accept the costmap as input. A costmap is an essential input to Move Base, representing the traversability of the environment in which the robot operates. In detail, If you have a robot surrounded by multiple obstacles, you can use the **A*** to compute an optimal trajectory. Additionally, human behavior trajectory data can be collected and used to refine the robot's path planning.

4.3 Methods for Collecting Human Behavior Data

There are two primary ways to obtain human trajectory data:

1. **Manual Data Collection:** An operator remotely controls the robot (teleoperation) to navigate within a human environment, recording paths that humans typically follow.
2. **Mathematical Path Modification:** If manual data collection is not feasible or you don't want to spend your time collecting this, you can apply curve shifting or curve modification techniques to adjust the trajectory generated by A* geometrically and mathematically. However, this method is rigid and lacks scalability.

So typically, people prefer to use teleoperation to control the robot in human environments in order to collect human trajectory data.

4.4 Optimazation

We collected a large amount of trajectory data and integrated it into a pipeline, which was then used to optimize robot navigation. During task execution, we obtain a top-down view to generate the Costmap. At each time step, we record the following information:

- The robot's current position.
- A snapshot of the environment.
- The target position.

Then, we run the **A*** and adjust the path weights based on the collected trajectory data:

- If a region is primarily occupied by human activity, it is assigned a higher cost value.
- The path computed by A* is further modified to align with these weight assignments.

Finally, when the adjusted **Costmap** is passed to **Move Base**, it automatically selects a path that better conforms to human trajectories. As shown in the figure 3.

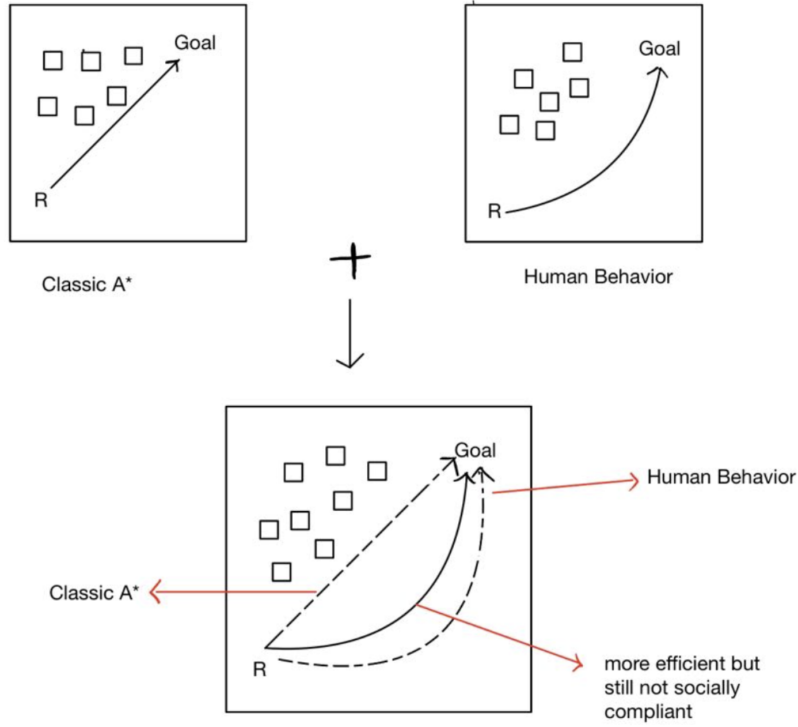


Figure 3: Costmap Layering-The Relative Optimal Path Obtained After A* and Human Behavior

This method is not a generalizable solution, as it may not adapt to all environments. However, it is the simplest and most effective approach, and it does not require the use of neural networks. In contrast, neural network-based learning methods heavily depend on the convergence of the model, meaning that if training is unstable, we may not obtain a desirable result.

Therefore, many engineers prefer Costmap-based methods because they do not rely on learning(neural network) and offer greater stability. This approach is known as Costmap Layering, where the main idea is to process data at different levels to generate a Costmap and pass it to Move Base for path planning. In this way, the robot can select more suitable paths based on the optimized Costmap, enhancing its navigation performance in social environments.

5 Trajectory Prediction

5.1 Research Development in Trajectory Prediction

5.1.1 2016: Social LSTM

This paper[1] proposes an LSTM-based model for pedestrian trajectory prediction, outperforming traditional methods and state-of-the-art approaches by learning general human movement patterns from past positions. It has demonstrated success on several public datasets, showing its ability to handle pedestrian trajectory prediction tasks with strong adaptability.

5.1.2 2018: Social GAN

This paper[2] introduces a method that combines recurrent sequence models and generative adversarial networks. By incorporating a novel pooling mechanism and diversity loss, it successfully predicts multimodal

and socially plausible human motion paths. The approach outperforms existing methods in terms of accuracy, collision avoidance, and computational complexity. This further enhances the generative capabilities of trajectory prediction, enabling the generation of more natural human trajectories.

5.1.3 2018-2019: SoPhie Framework

The SoPhie[3] framework aims to address the path prediction problem in multi-agent scenarios, with applications in autonomous platforms such as self-driving cars and social robots. SoPhie combines social attention and physical attention mechanisms, successfully modeling both physical and social interactions by leveraging the path history of all agents and scene context information. The framework also uses Generative Adversarial Networks (GANs) to generate more realistic path predictions and capture the uncertainty of future paths. Through these mechanisms, SoPhie achieves state-of-the-art performance on multiple trajectory forecasting benchmarks.

5.1.4 2020-2024: Research Boom

Trajectory prediction has become a highly active research field, with approximately 20 related papers published annually. As the datasets and evaluation standards have become fixed, most research focuses on adjusting architectures and optimizing data processing to improve model performance on established datasets. With the ongoing research, new methods and techniques continue to emerge, driving the continuous development of the trajectory prediction field.

However, when these methods are applied to robot navigation, their transferability and generalization is very poor. Many algorithms perform well in environments based on driving datasets, but perform poorly in actual robot navigation.

5.2 Trajectory Forecasting

Trajectory prediction is a very popular topic, especially in the field of autonomous driving. The trajectory data $\tau_{t-H:t}^{1:k}$ from the time period between $t-H$ and t is used to provide historical information. After passing through the encoder-decoder model, the predicted future trajectory $\tau_{t:t+Z}^{1:k}$ during the time period from t to $t+Z$ can be obtained. To be more specific, at timestep t , $x_t^i \in \mathbb{R}^D$ represents the state of each agent, where D is the dimension of the state space. The historical trajectory of all agents from timestep $t-H$ to t is represented as:

$$x_{t-H:t}^{1:k} \in \mathbb{IR}^{(H+1) \times k \times D}$$

where $H+1$ is the number of timesteps (including timestep t), k is the number of agents, and D is the dimensionality of each agent’s state space. This captures the history of the agents’ positions, velocities, etc. Additional information at timestep t is added to the state of all agents as $I_t^{1:k}$. This additional information could include environmental factors like obstacles, other agents, maps, semantic information, etc., and helps to refine the trajectory prediction. The goal is to predict the future trajectory of all agents for the next T timesteps, denoted as:

$$y = x_{t+1:t+T}^{1:k} \in \mathbb{IR}^{T \times k \times D}$$

where T is the number of timesteps to predict, k is the number of agents, and D is the dimensionality of each agent’s state. This represents the distribution of the agents’ states over the next T timesteps.

For the encoder-decoder network, we can use LSTM or other structures to capture the temporal relationships in the time series. During the process of passing data through the encoder-decoder network, additional environmental information, such as obstacles, map information, and lane information, can also be input. This suggests that in the trajectory prediction task, the input data is multi-channel, meaning that different types of information can be encoded into different channels. Therefore, in complex scenarios, the observation function can be written as:

$$O_i^t = \{x_i^t\} \cup \{x_j^t, j \in N(x_i^t)\} \cup M_i^t \quad (11)$$

Where M_i^t represents environmental information, such as semantic maps, etc.

However, trajectory prediction technology is still very complex. Currently, most applications are still concentrated in computer vision tasks, with more focus on algorithms and benchmark datasets. Therefore, traditional trajectory prediction methods are more often used as benchmark tests in the field of computer vision, rather than being applied to the real world.

5.3 Social Compliance and the Jerk Agent Problem

In conclusion, social compliance refers to the idea that the more closely a robot’s behavior aligns with human behavior in social environments, the more socially compliant it is. However, there is currently no objective standard to measure whether a robot truly exhibits social compliance. This is due to the involvement of humans and their differing perceptions of what social compliance is. Therefore, we can choose our preferred approach to social compliance (such as costmap layering).

In addition to algorithms, there are other requirements for achieving social compliance:

1. A standard formal evaluation system, which means we need to rethink the existing evaluation framework.
2. A high-fidelity, rich, and lightweight simulator to generate a wider variety of driving scenarios.

An interesting problem arises when an agent behaves unpredictably (jerk agent). In this case, existing social navigation methods may completely fail. That is to say, you can design nice and fancy models, but if you encounter an agent that does not conform to your rules, the performance of all models will be significantly reduced. For example, when a robot and a human are both heading towards the same door and trying to pass through at the same time, if the human behaves in an extremely non-compliant way, the safest and most effective approach is to avoid colliding with this jerk agent. However, such "jerk agents" are very rare, which is why most datasets do not capture these extreme behaviors. Although such behaviors exist in the real world, the lack of corresponding datasets causes models to perform poorly in such extreme situations. For instance, our trajectory prediction models are based on traffic datasets from the United States, but when these models are applied to some developing countries, they do not perform well.

References

- [1] Alexandre Alahi, Kratarth Goel, Vignesh Ramanathan, Alexandre Robicquet, Li Fei-Fei, and Silvio Savarese. Social lstm: Human trajectory prediction in crowded spaces. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 961–971, 2016.
- [2] Agrim Gupta, Justin Johnson, Li Fei-Fei, Silvio Savarese, and Alexandre Alahi. Social gan: Socially acceptable trajectories with generative adversarial networks. *2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 2255–2264, 2018.
- [3] Amir Sadeghian, Vineet Kosaraju, Ali Reza Sadeghian, Noriaki Hirose, and Silvio Savarese. Sophie: An attentive gan for predicting paths compliant to social and physical constraints. *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1349–1358, 2018.